

Unified Asset & Operations Brain

ET - AI Hackathon (Problem Statement 8) — Implementation Plan

Implementation Plan: Unified Asset & Operations Brain (Industrial Knowledge Intelligence)

This document outlines the design, architecture, and step-by-step plan for building the **Unified Asset & Operations Brain**, a prototype for the ET - AI Hackathon (Problem Statement 8). The project will be built in the workspace directory `C:\Users\dubey\Desktop\ET_AI`.

User Review Required

IMPORTANT:

API Key Selection & LLM Providers

We need an LLM API to handle entity extraction, semantic answers (RAG), and root cause analysis.

I propose supporting **Gemini API** (using Google GenAI SDK) as the default (fast, cost-effective, generous free tier), while also supporting **Anthropic Claude API** and **OpenAI API** as configurable fallbacks via environment variables.

Please let me know which API key(s) you have available.

[!WARNING]

PowerShell Execution Policy and Node Commands

Your system has PowerShell script execution disabled, which prevents directly running `npm` commands.

Solution: I will use `npm.cmd` and `npx.cmd` explicitly in all commands to bypass this restriction, which works perfectly. You won't need to change system settings.

[!NOTE]

Embedding & Vector Store Fallback Strategy

Windows environments sometimes lack the C++ build tools required to compile `chromadb` or download heavy local libraries.

Solution: We will build a unified Python storage module that:

- Attempts to use `chromadb` for vector storage.
- If `chromadb` fails to install or load, automatically falls back to a custom **SQLite+NumPy-based Vector Store** (which uses a simple SQLite table to store raw text chunks and float array embeddings, with a pure-Python cosine-similarity implementation). This guarantees the app will run out-of-the-box on any machine.

3. For embeddings, we will support both a lightweight local model (like `all-MiniLM-L6-v2` via `sentence-transformers`) and an API-based embedding (Gemini/OpenAI/Claude).

Open Questions

1. **Which LLM API Key will you use for the hackathon?**
2. Option A: Google Gemini API (Recommended: easy setup, high rate limits).
3. Option B: Anthropic Claude API.
4. Option C: OpenAI API.
5. Option D: A mock/local mode (where we simulate LLM responses for offline debugging if no key is available immediately).
6. **Do you have a preference for the visual design?**
7. I plan to build a sleek **industrial-dark mode dashboard** (using Outfit or Inter typography, dark slate backgrounds, glassmorphism card components, and status colors like Amber/Blue/Green for active/warning/nominal statuses) that looks highly professional and mimics control room displays. Does this sound good to you?

Proposed Changes

We will build the project inside the `C:\Users\dubey\Desktop\ET_AI` directory, structuring it as a monorepo containing `backend/` and `frontend/` directories.

```
c:\Users\dubey\Desktop\ET_AI\  
■■■ backend/  
■   ■■■ ingestion/  
■   ■   ■■■ parse_documents.py      # Extract text from PDF, CSV, XLSX, TXT  
■   ■   ■■■ chunk_text.py          # Chunking text into ~500-word segments  
■   ■   ■■■ extract_entities.py     # Extract equipment IDs, relationships (triples) via LLM  
■   ■■■ storage/  
■   ■   ■■■ vector_store.py         # Vector Store (ChromaDB + SQLite fallback)  
■   ■   ■■■ knowledge_graph.py      # SQLite Knowledge Graph (entities + triples)  
■   ■■■ rag/  
■   ■   ■■■ retriever.py            # Retrieve semantically relevant chunks  
■   ■   ■■■ query_engine.py         # Formulate RAG answers with citations  
■   ■■■ agents/  
■   ■   ■■■ rca_agent.py            # Root Cause Analysis agent logic  
■   ■■■ api/  
■   ■   ■■■ main.py                 # FastAPI server exposing REST endpoints  
■   ■■■ requirements.txt            # Python dependencies
```

```

■   ■■■ run.py                                # Main entrypoint to start backend
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ components/
■   ■   ■   ■■■ Sidebar.jsx                  # App navigation
■   ■   ■   ■■■ Dashboard.jsx                # Stats, quick insights, and charts
■   ■   ■   ■■■ DocumentManager.jsx          # Document list & upload form
■   ■   ■   ■■■ ChatCopilot.jsx              # Chat RAG UI with citation panel
■   ■   ■   ■■■ RcaAgent.jsx                  # Equipment selection & RCA timeline report
■   ■   ■■■ App.jsx                           # Main page assembler & state manager
■   ■   ■■■ index.css                         # Custom global styles (Tailwind or custom CSS)
■   ■   ■■■ main.jsx
■   ■■■ index.html
■   ■■■ vite.config.js
■   ■■■ package.json
■■■ sample_documents/                          # Move the user's sample documents here

```

Backend Component (Python + FastAPI)

The backend will handle parsing, ingestion, database management, and LLM orchestration. We will create a virtual environment (.venv) to run it.

[NEW] [requirements.txt](#)

Define backend packages:

- fastapi, uvicorn (API server)
- pydantic (Data schemas)
- pdfplumber, pypdf (PDF extraction)
- pandas, openpyxl (Spreadsheets)
- sentence-transformers (Local embeddings option)
- numpy (For fallback vector operations)
- google-genai, anthropic, openai (LLM SDKs)
- python-dotenv (Configuration management)
- chromadb (Vector DB)

[NEW] [knowledge_graph.py](#)

A lightweight relational store in SQLite (knowledge_graph.db) with tables:

- documents: id (hash), name, type, uploaded_at, status
- entities: id (e.g. "C-102"), name, type (EQUIPMENT, PERSON, SOP, REPORT)
- triples: id (integer auto), subject (entity_id), predicate (relationship), object (entity_id), source_doc (document_id)

[NEW] [vector_store.py](#)

Abstract interface supporting both ChromaDB and SQLite fallback:

- `add_chunks(chunks: List[str], metadatas: List[dict])`
- `query(text: str, n_results: int) -> List[dict]`

[NEW] [parse_documents.py](#)

Text extraction logic:

- Reads TXT files cleanly.
- Reads PDFs using `pdfplumber` (text and tables).
- Reads spreadsheets (CSV/XLSX) and formats them into text representations (Markdown tables) for easier vector chunking.

[NEW] [extract_entities.py](#)

Uses the LLM (Gemini/Claude) with structured outputs (JSON schema) to extract entities and triples from document text (e.g. "C-102" is EQUIPMENT, "WO-2023-0847" is REPORT, relationship: C-102 -> MAINTAINED_IN -> WO-2023-0847).

[NEW] [query_engine.py](#)

The core RAG module. When a query is made:

1. Embeds query, performs vector search for text chunks.
2. Extracts potential entity keywords from query, searches the SQLite knowledge graph for matching entities and their neighborhood triples.
3. Formulates a prompt combining the question, semantic text chunks, and relational triples.
4. LLM answers and cites specific source documents.

[NEW] [rca_agent.py](#)

Specialized Maintenance & Root Cause Analysis Agent:

1. Queries all documents and triples linked to `equipment_id` (e.g., "C-102").
2. Constructs a chronological timeline of maintenance logs, inspection reports, and operating parameters.
3. Prompts the LLM to act as a Principal Reliability Engineer:
 - Identify recurring failure modes (e.g., "monsoon season moisture entry").
 - Highlight violated SOPs/procedures.
 - Suggest root cause.
 - Issue actionable recommendations and predictive alerts.

[NEW] [main.py](#)

FastAPI routes mapping to:

- `POST /api/ingest/file` (Single file upload)
- `POST /api/ingest/initialize` (Batch ingest files in `sample_documents/` folder)
- `POST /api/ask` (Payload: `{question: str}` -> Returns `{answer: str, citations: list, confidence: float}`)
- `GET /api/entities/{id}` (Returns details and relationship triples for a specific entity)

- `POST /api/rca` (Payload: `{equipment_id: str}` -> Returns `{timeline: list, report: str, recommendations: list}`)
 - `GET /api/documents` (List all documents and their ingestion status)
-

Frontend Component (React + Vite + Tailwind CSS)

We will initialize a modern React app inside the `frontend/` folder. Since Tailwind CSS provides an excellent developer experience for building highly responsive, beautiful interfaces quickly, we will use it alongside vanilla CSS.

[NEW] [App.jsx](#)

State container and page manager. Contains views for:

1. **Dashboard:** Industrial stats (total documents, KG nodes, active alerts).
2. **Chat Copilot:** The conversational RAG system.
3. **RCA Agent:** Dedicated panel to input an Equipment ID and see a timeline and generated report.
4. **Document Library:** Lists all ingested documents with statuses (Ingested, Parsing, Failed) and handles manual uploads.

[NEW] [ChatCopilot.jsx](#)

A split-screen view:

- **Left Panel:** Conversational chat interface. When you hover over a citation in the LLM's answer (like `[W0-2023-0847]`), it highlights the source document.
- **Right Panel:** Shows the detailed context retrieved (text chunks + knowledge graph relationships) for the current response.

[NEW] [RcaAgent.jsx](#)

Interactive reliability analysis tool:

- Search or dropdown selector for equipment IDs (e.g. `C-102`, `PRV-88`).
 - Vertical timeline showing maintenance events, inspections, and logs chronological order.
 - Sleek markdown report display showing Root Cause Analysis and suggestions.
-

Verification Plan

We will test the system locally using the synthetic documents in the workspace.

Automated Tests

We will write a python test runner `verify_pipeline.py` in the `backend/` directory to run verification checks without loading the full web app:

- Verify text extraction of the sample documents.
- Verify vector storage and retrieval.
- Verify SQLite knowledge graph CRUD operations.

- Verify LLM connectivity and response formatting.

Manual Verification

1. **Bulk Ingestion:** Trigger `/api/ingest/initialize` to parse all sample documents.
2. **RAG Copilot Verification:** Ask queries such as:
3. *"What was the issue reported with C-102 in July 2023?"* (Should retrieve `maintenance_log_001.txt` and report valve wear and moisture ingress).
4. *"When is the next inspection due for PRV-88?"* (Should retrieve `inspection_report_ir2023_055.txt` and answer March 12, 2024).
5. **RCA Analysis Verification:** Run the RCA agent on `C-102` or `Pump 4B` and check if it successfully summarizes the maintenance history timeline and provides root cause hypotheses.